

A lightweight framework for describing software practices

Diana Kirk*, Ewan Tempero

Department of Computer Science, University of Auckland, Private Bag 92019, Auckland, New Zealand

ARTICLE INFO

Article history:

Received 25 March 2011

Received in revised form

12 September 2011

Accepted 12 September 2011

Available online 19 September 2011

Keywords:

Software practices

Software organisations

Industry case study

Software process discovery

Research framework

ABSTRACT

In order to maximise software project outcomes, software organisations adapt development methodologies and implement practices in a way that is appropriate for project contexts. This suggests that 'Best Practice' is context-dependent. To better understand the contextual nature of best practice, we want to explore how organisations actually go about achieving software objectives. We require a research framework that captures this information in a way that makes no assumptions about practices and that is descriptive in nature. We have developed a framework based on the perspective that practices exist to meet specific objectives. We have experimented with the framework by using it to capture the practices of three New Zealand organisations and by application to an idealised XP process. Our capture of organisational practices revealed interesting mechanisms for further study, including a dependence upon informal practices linked with strong communication and the idea of 'push' versus 'pull' for information elicitation. Our capture of XP exposed some context-dependent risks.

© 2011 Elsevier Inc. All rights reserved.

1. Introduction

All software organisations carry out the same kinds of activities when creating and enhancing software-intensive products and services, but these activities are implemented in many different ways. For example, in some organisations, analysts carry out requirements elicitation in a formal way and capture requirements in software specification documents from which developers implement the product while in others developers speak directly with clients and are involved in both requirements elicitation and product implementation. In many cases, the more informal approach appears to work as well as the formal, for example, in small organisations. Indeed, it is likely that, for such organisations, implementing a more formal approach would be inappropriate and introduce unnecessary overhead. Although the industry promotes formality, there is increasing agreement that in these cases activities are best tailored according to an organisation's business objectives and contexts (Basili et al., 1994; Dawson et al., 2004; MacCormack et al., 2003; Munch, 2005). Recent research suggests that many organisations successfully combine agile and traditional practices to achieve the benefits of both (Baskerville et al., 2011).

The long-term goal of our research is to better understand how successful organisations adapt software development activities to suit their own, specific context. If we are able to successfully model an organisation with respect to activities and contexts, we may

uncover useful trends and patterns that can be used to categorise organisations along a number of key dimensions. This understanding would provide us with a means of more effectively predicting performance and advising suitable improvement directions that are context-specific.

Representing how groups develop software typically involves assessing software development organisations according to the externally architected assessment models such as CMMI (Chrissis et al., 2007) or ISO/IEC 15504 (ISO, 2004/2006). This approach does not suit our needs because many software development organisations, particularly small-to-medium enterprises (SMEs), do not follow such process models (Cater-Steel et al., 2004; Staples et al., 2007) and so this approach would help expose what the organisations *do not do*, but would not help us understand their mechanisms for success. The root of the problem is that such models are *prescriptive* in nature and inherently contain many assumptions about what constitutes 'best practice'. We want a *descriptive* model that makes no assumptions but rather supports uncovering what organisations *actually do* as opposed to what they *should do*.

Simply asking organisation members in a general way about methodologies adopted also is ineffective, because we have found that often an organisation will report following a particular methodology, but in fact only part of the methodology is adopted or adoption is not true to the intent of the methodology. If implementation of an agile methodology is claimed, the problem is exacerbated because the vague nature of the term *agile* means that the claim of 'agile practices' could mean practically anything (Baskerville et al., 2011; Livari and Livari, 2011). As illustration, consider a study to determine if Scrum is more productive than XP. Our approach may be to study productivity in a sample of organisations

* Corresponding author. Tel.: +64 9 373 7999; fax: +64 9 373 7453.

E-mail addresses: dianakirk@acm.org (D. Kirk), e.tempero@cs.auckland.ac.nz (E. Tempero).

that implement Scrum and a sample that implement XP. However, we cannot be sure that each organisation is true to the Scrum or XP approach and without such understanding our study is meaningless. We need to effectively capture what actions actually take place in a way that addresses concerns like these.

As a first step towards meeting our long-term goal, we require a mechanism for capturing information about implemented practices that will support exposure of some key differentiating aspects between organisations. Our first attempts at capture, based on the ISO/IEC 15504-5 process reference model (ISO, 2004/2006), were highly unsuccessful. We found that the prescriptive nature of the standard was too restrictive, in that there were situations where we could not be sure we had captured what a company was doing, because it did not appear to fit in the standard. In addition, companies often did not use the same terminology as the standard and this led to a frustrating and ineffective experience for both researcher and participant.

We realised that we needed a change of focus. We have elsewhere suggested that an appropriate approach for understanding software practice is to 'lift' the focus from 'choosing practices from a pre-defined set' to 'identifying project objectives and selecting activities to meet these objectives' (Kirk et al., 2009). Our contribution in this paper is a research framework based on this focus. Practices are collated in this framework and are effectively treated as the factors (independent variables) that potentially affect project outcomes, such as functionality and quality (the dependent variables). The framework provides a means of establishing how organisations apply software practices to meet specific objectives. As such, it may be used to support research questions relating to, for example, categorising organisations, context-dependent best practice and risk and issue exposure.

In this paper, we overview the study that led to the development of our research framework and present the resulting framework. As the framework supports different kinds of research question, we have chosen an evidence-based approach to evaluation. We describe three studies, the first a case study involving three software organisations in Auckland, New Zealand (NZ), and the others a theoretical examination of risk in an both 'ideal' and implemented XP practices.

The rest of this paper is organised as follows. In Section 2, we overview other studies aimed at understanding how software organisations operate. In Section 3, we present the details of our study. In Section 4, we present the proposed research framework. In Section 5, we describe the three studies using the framework and discuss insights gained. In Section 6, we discuss the evidence as it relates to the kinds of questions we need to ask about our framework. Finally, we present our conclusions and we discuss future work in Section 7.

2. Related work

A number of empirical studies have been done that seek to understand the operation of software development organisations. Many of these studies have as their main goal evaluating or assessing the organisation's operation against a particular model or set of practices. One study with a similar goal to ours is by Coleman (2005). The author tries to determine why software SMEs do not appear to be using standard process models. He interviewed 15 CTOs from SMEs in Ireland and reported their reasons for not using standard models. His main finding was that this was due to the perceived overhead of such process models. While Coleman's goal is similar to ours, he reports why they do not use some models rather than what they do to be successful.

Cusumano et al. conducted a survey to determine what international differences there are in how software is developed (Cusumano et al., 2003). Their survey gathered data on the use of specific practices from 104 projects from organisations across the world. While they limited their survey to a set of practices, their results do indicate possible differences between organisations in different countries, and this may affect the generalisability of the results described in this paper.

There have been several studies evaluating different aspects of SPI programs. For example, Cater-Steel, Toleman and Rout present the results of a study of 22 Australian SMEs evaluated using an assessment method based on ISO/IEC 15504 (Cater-Steel et al., 2004). Their goal was to determine the extent to which these companies meet the standard and to provide advice on software process improvement. They identified three common 'problems', one of which was "competent staff were relied upon rather than documented processes." Staples et al. did a study to identify reasons why organisations do not adopt CMMI (Staples et al., 2007). Their results indicated a perception that CMMI was not for "small" organisations, and a perception that CMMI was too costly to adopt.

Other empirical studies have evaluated the predictive power of SPI standards. One example is by El Emam and Garro, who looked at the part of ISO/IEC 15504 relating to requirements engineering, and found evidence to support that higher capability for requirements engineering as assessed by this standard related to increased productivity for large software development organisations, but found not such evidence in the case of small organisations (El Emam and Garro, 2000). All of the organisations in our study would be regarded as small.

Another category of studies examines software development practices on a region-specific basis. However they also assess against specific software development models or sets of practices. This is true of the Cusumano et al. (2003) and Cater-Steel et al. (2004) studies mentioned above. Other examples include: Dutta et al. (1999) surveyed 397 groups in 20 European countries to determine the degree to which formal models such as SPICE and CMMI are used; Sison et al. (2006) surveyed organisations in five ASEAN countries (Malaysia, Philippines, Singapore, Thailand and Vietnam) to better understand practices used in that region. Their survey was based on published best-practice, such as SEI's SW-CMM Maturity Questionnaire for Level 2. Our study is specific to New Zealand, however that was a consequence of limited resources rather than a specific goal.

The studies described above are valuable to understand what issues may be associated with particular models, however they do not directly help us understand why those organisations that do not appear to use specific models nevertheless also appear successful.

3. Motivation for framework

Our motivation for studying the practices used in NZ organisations comes from our perception, based on anecdotal evidence and our own experience, that by and large these organisations do not use formal models such as CMMI or SPICE, or sets of practices such as those used in the Agile community. Despite this these organisations seem successful, leading us to want to understand why. Although our findings will relate to NZ organisations, we note that many other countries have a demographic similar to NZ, i.e. mainly SMEs with a tendency towards creative endeavour (Bach, 1995; Laporte et al., 2008; McCaffery et al., 2008; MoRST, 2009; Petterssen et al., 2008), and so we believe that findings will be of general interest to the software community.

Our original intent was to identify possible explanations from the literature and verify them with quantitative studies of companies in New Zealand. However, we found that most empirical

studies reported in the literature evaluate an organisation's operation against a particular model or set of practices, rather than capture how organisations actually work. This led us to realise that we first needed a research model to help us find out what might be going on, and to support creation of hypotheses for further testing. We planned to refine a model based on the ISO/IEC 15504-5 process reference model (ISO, 2004/2006) by conducting a trial with two software organisations and then testing the refined model by applying to a further three organisations. Our experiences during the trial caused us to rethink what we needed to achieve.

We overview the research approach in Section 3.1. In Section 3.2, we briefly describe the problems found during the trial and present the revised research questions. In Section 4, we present our framework.

3.1. Research description

The research questions for our original planned study related to understanding software practices in NZ organisations. It is widely acknowledged that software projects are complex as a result of the mix of people, changing technologies and the difficulties involved in defining software products. The case study is an established approach for gaining insights in complex environments (Yin, 1994), so we implemented an *exploratory multiple-case study*. As the research relates to understanding how organisations carry out practices, the unit of analysis was a *software organisation*. As the practices carried out were the focus of our attention, the sub-unit of analysis was *the set of practices carried out by a software organisation when creating software products or services*.

The organisations approached for participation were selected from a list of organisations currently involved in a government funded Software Process and Product Improvement (SPPI) project of which this research is a constituent. The reason for this was pragmatism. In the first instance, our research was exploratory in nature with the aim of creating a suitable research model, and so issues of randomness do not occur. In addition, the organisations available varied in size and application area and thus provided a suitable sample space to support our desire to capture the practices of different kinds of software organisation.

The main contact for each organisation, the *manager* for our study, was the CEO or owner (for smaller organisations) or a senior manager with authority to make decisions relating to the software development function (for larger organisations). The names of individuals from within organisations were provided by the manager at the end of the first interview. We believed the manager would at this point have a clear understanding of what we hoped to achieve and would be more likely to provide a realistic cross-section of participants rather than providing only participants who would give an optimistic version of organisational functioning. Talking to employees other than those provided by management was not considered both for formal ethical reasons and because we would view this as a serious breach of trust. However, we did mitigate in two ways. We asked the manager to provide participants covering a range of activities at a range of levels, i.e. not all managers and team leaders, who might present an idealised version of the truth. We also reminded participants at the start of interviews that all information given was confidential and would not be passed back to management (as required by the University Ethics Approval process). We indeed found that comments were both positive and negative.

We interviewed two or three persons with varying roles plus the manager from each organisation. This number was also based on pragmatism. We thought three employees in different roles would provide a sufficient breadth of viewpoints to support the consolidation of reports about practices and would expose inconsistencies. We requested from the manager 'at least three' persons and were

provided with three names from each of the participating organisations.

As suggested by Yin (1994), we focused the research by addressing specific questions, i.e. our research questions. As data collection instrument and protocol, we applied a semi-structured interview based on a questionnaire (for the manager only) and a *practices template* for all other participants, based on the reference model (ISO, 2004/2006). Both were tested and evolved during a trial involving volunteers from two development groups from within a fourth organisation.

We implemented a number of standard research protocols, for example, relating to permission, anonymity and confidentiality.

3.2. Trial

There were three main drivers for the design of our study. Anticipating that NZ organisations would be unwilling to spend a lot of time on our study, our research approach would have to be *lightweight*. Organisations are reluctant to have their staff involved in non-core activities and the more time it takes, the less likely they would be willing to participate in our study. In order to minimise the impact of our study on an organisation, we set 1 h as the maximum amount of time to spend with any one person, that being an amount of time we thought organisations would be willing to contribute.

The second driver was, we needed to be sure we elicited the same kind of information from each organisation, that is, we did not ask different kinds of questions of different organisations. This was complicated by the fact that different individuals would know different aspects of their organisation. That meant we needed the model we used to be quite well structured, a property we refer to as *repeatability*.

At the same time, we needed to be confident that we had fully captured the practices of an organisation. If we missed some practices that were being used we would mis-represent what was in fact was going on. This meant our study had to cover all the possibilities we might encounter, a property we refer to as *completeness*.

We learned that participants tended not to 'stick to the script' and would offer comments as they occurred to them that were not necessarily relevant to the current topic. That left us either asking them to repeat the information when we got to the relevant topic or try to capture it outside the process we were following. This convinced us that any framework we used had to be *flexible*, in that it had to give freedom to participants to provide information when it occurred to them and answer questions how they chose.

During the trial, we found that our original approach, based on the ISO reference model, was unsuccessful in supporting the above goals. The main barriers related to terminology and model structure. In the first instance, participants often did not understand phrases such as 'criteria for the integrated software to demonstrate compliance with software requirements' (ISO, 2004/2006). The need to ask very fine-grained questions and explain terminology resulted in us being unable to fit in the 1 h limit, risking either repeatability or being too heavyweight. We also found that participants wanted to discuss all aspects of an activity at the same time and so the separation of engineering and support processes such as documentation led to an interview experience that was frustrating for both interviewer and participant. The lack of flexibility in the model left us unsuccessful in achieving our objectives. Further details on the study are available in Kirk and Tempero (Submitted for publication).

Our problems during the trial caused us to question if our goals were even possible to achieve. This led to the following revised research questions for our study.

		Content	Quality
Define	Roadmap	X	X
	Scope	X	X
Make	Design	X	X
	Implement	X	X
	Integrate	X	X
Deliver	Release	X	X
	Support	X	X

Fig. 1. Basic research framework.

3.3. Research questions

1. Can we establish a useful mechanism for describing practices as implemented by software organisations?
2. Does the use of this mechanism yield any insights that might become the basis of more formal study.

As a result of the changed questions, we had to reconsider our study approach. We first created a framework based on earlier work and the experiences described above. We describe this framework in Section 4. We then applied the framework in a number of different kinds of research context in order to evaluate its usefulness as a mechanism for describing practices and yielding insights. We discuss framework evaluation in detail in Section 5.

4. Research framework

In this Section, we describe the two dimensions that comprise the basic framework structure and follow with a description of the extensions to the basic framework that resulted from the case studies. We then overview some possible research scenarios where the use of the framework might yield insights and avenues for further exploration. We finally describe how to use the framework for describing practices.

4.1. The basic framework

As a result of the problems we experienced in the early attempts, we instead took an approach focusing on what organisations *need to achieve* rather than on the processes required to achieve it. Thus, instead of working through a list of practices that the organisation might have, or asking for a list of practices used by the organisation and us having to try to map them on to some standard list, we structured our framework according to some key needs that must be addressed when software is developed. This approach is based on some earlier work, in which we highlighted the need to view software process activities as being there for a reason, that is, to support the achievement of some business objective (Kirk et al., 2009). For example, testing is carried out with the aim of ensuring quality objectives are met and prototyping is often applied to help ensure the software product exhibits appropriate functionality.

The framework is thus based on the viewpoint that the software practices implemented by an organisation can be understood only in the context of the organisation's key objectives. There are many possible such objectives, for example, relating to quality, functionality, stakeholder and user satisfaction and cost. For reasons of pragmatism, we focus on the objectives of *content* and *quality*. Other objectives will be added to the framework in future work. In Fig. 1, we show the basic research framework structure with the objectives along the top.

Content refers to the expected features and functionality in the software product *from the perspective of the customer(s)*. A

product delivered with missing or incorrect functionality is, from our perspective, deficient in *Content*. Examples of the kinds of activities carried out to ensure correct *Content* are formal requirements tracking, prototypes discussed with the client, separate test team who test based on requirements documents, inspections and reviews focused on checking for correct functionality, the use of a wiki to store information about features and how these should work, having a subject area expert on the team.

Quality refers to agreed stakeholder expectations *other than* those relating directly to features and functionality. For example, expectations may relate to efficiency, security or how easy it is to access the functionality. A product delivered with missing or incorrect functionality but with efficiency and usability expectations met is, from our perspective, high in *Quality*. Examples of the kinds of activities carried out to ensure appropriate *Quality* include formal requirements documents that include a section for non-functional requirements, pair programming where one programmer constantly checks for injection of defects into the code, a test environment that is the same as, or similar to, a client's environment, adherence to a strict build submission process.

The above perspective is not the commonly understood idea that 'high Quality' infers that both functional and non-functional expectations are met. A 'good' testing approach is perceived as being one that tests for both. Our reasons for separating these aspects are grounded in the observation that many activities assumed to address both aspects equally do not, in fact, do so. This means that we often make assumptions about what is being done. For example, specification techniques such as 'Use Cases', and 'User Stories' do not inherently consider non-functional requirements. This often results in a belief that the 'product has been specified' when it is in fact only partially specified. In a similar way, we might assume that a product testing approach that is effective for finding defects is effectively addressing both *Content* and *Quality*. However, if the test teams source expected functionality from a wiki where features are described by developers, the testing activity may be extremely ineffective at uncovering *Content* issues because the developers may have misunderstood user expectations. The 'dimensional' approach provides us with a means of clarifying what is actually being done.

We also note that:

- 'Content' is often used to mean the aggregation of all items included in the delivered product, for example, binaries, persistent data files, configuration files and documentation. This is a different meaning from the one in our framework. From our perspective, the management of such an aggregation may constitute a practice that affects either delivered *Content* (for example, if an incorrect configuration file is delivered) or *Quality* (for example, if an older, more buggy version of a binary is delivered).
- During development, developers often test the 'functionality' implemented in their code. Unless these tests directly check expected features and/or functionality from the perspective of the client, the developers are addressing *Quality* and not *Content*, i.e. are checking the integrity of their code.

In order to help focus questions and organise information provided by people in different kinds of roles, we next introduced an orthogonal categorisation. It is based on what must be achieved at a high level, i.e. on the fact that, when producing any product, the producing organisation must

- Define what is to be made.
- Make it.
- Deliver it.

In Fig. 1, these functional categories are shown down the left-hand side. We extended the above categories to include what we believe are the main sub-categories for software. Functional sub-categories for *Define* relate to approaches to long-term strategic (*Roadmapping*) and short-term project (*Scoping*) product planning. Categories for *Make* include what appear to be the most generally accepted elements of making software, i.e. choosing a suitable solution (*Design*), implementing the pieces of the solution (*Implement*) and putting it all together (*Integrate*). For *Deliver*, we include aspects relating to the interface between development and client organisations, namely transfer of the product (*Release*) and the product as part of the target system (*Support*).

According to our framework, these functions provide a sufficient and exhaustive set for software development activities. Note, however, that the intention is not to impose any ordering of the functions. The intent of the framework is to provide a structure for asking questions and collecting data in a repeatable, flexible and complete way, with no assumptions about practice implementations. For example, with *Make*, there is no expectation that practices associated with *Design* and *Implement* need be separate.

Each cell of the framework is a repository for information about the practices that relate to the function (row) and objective (column) for that cell. In Fig. 1, we show an X in each cell to denote that these practices will vary according to context. To illustrate some possible contexts and practices, we present some examples.

1. The owner of a small start-up company brainstorms his idea for a new software product with an experienced developer. He sketches some diagrams for the developer to work from. Both are involved in the *Define:Roadmap* function for the *Content* objective. The discussions include the need for scalability and efficiency and so the discussion activity also addresses the *Define:Roadmap* function for the *Quality* objective.
2. The Product Manager for a software organisation that delivers to both the US and Asia-Pacific marketplace bases all decisions about product introduction and upgrades on an extensive market analysis which includes consideration of both functionality (*Content* objective) and quality (*Quality* objective). Resulting decisions are then captured in formal requirements specifications which are signed off by the Business Analysis team. Activities for *Define:Roadmap* for both objectives include 'Formally analyse market' and 'Formally document and sign off requirements'.
3. A developer who unit tests a code module prior to submission to the build is involved in the function *Make:Implement* for the *Quality* objective, i.e. the tests are aimed at exposing defect injection and thus aim to improve the quality of the delivered product.
4. A development lead works for a company that produces custom software. The company has a policy of client involvement in the process and always assigns a senior developer to act as the primary interface to each client. At project start-up, the client describes the kind of product he wants and some key desired features to the development team. The development lead then meets weekly with the client and uses prototypes to help consolidate functionality (*Content* objective). Functionality is captured in Use Cases and these are passed to the team for implementation.

The development lead is involved in *Define:Scope*, with activities including 'Prototyping', 'Regularly meet with client' and 'Document functionality in Use Cases'. He monitors development by regular, but informal meetings with the development team and is thus involved in the *Make* function. He also participates in the *Deliver:Release* function because, before releasing a product, he checks the status of agreed Use Cases and the defects log and discusses status with the client. As specific quality expectations

are not discussed, the *Quality* objective is addressed only by the defect log.

5. A developer is involved in design reviews held with the intention of checking design quality. The activity addresses the *Make:Design* function for the *Quality* objective. If the design review also checks for compliance against requirements, the activity would also address the *Content* objective. We observe that the same practice might appear in more than one cell. However, although the practice might be included in more than one place, its *efficacy* according to the viewpoint of the practitioners might be quite different. For example, the design review might be reported as being extremely effective as a quality assurance mechanism, but weak as a content assurance mechanism due to the unavailability of comprehensive requirements documentation.
6. A test team validating the product prior to delivery to a client is involved in the function *Make:Integrate*, (i.e. is working with the integrated product). It is likely that they are addressing objectives for both *Content* (testing functionality is as expected) and *Quality* (checking for defects and non-functional requirements). However, they may be addressing the two objectives with varying levels of effectiveness. The team may be extremely experienced, motivated and proficient at finding defects, but if they do not have good access to descriptions of what the product should do, they may be ineffective when addressing the *Content* objective.
7. A developer enhancing a delivered product is involved in the function *Make:Implement* (and perhaps also *Make:Design*). The *Content* objective may be addressed by having a description of expected functionality and the *Quality* objective by a testing standard for patches. Note that, for a defect resolution, the absence of the functionality description may result in the developer unknowingly modifying functionality, thus compromising the *Content* objective. If the developer provides a patch with the resolved defect to a client, (s)he is involved in the *Deliver:Release* function, i.e. there is now some relationship with the client.
8. A development lead is expected to liaise regularly with the client to ensure the delivered product continues to meet needs. The organisation have found in the past that clients have become unhappy because product effectiveness is reduced as a result of technology changes in the client's system. The organisation asks the development lead to monitor the situation. As (s)he is concerned with understanding the product in the client environment, (s)he is involved in *Deliver:Support*.
9. A team that provides first tier support for a product, i.e. troubleshoots reported product malfunction, is involved in *Deliver:Support*. If an issue is acknowledged as being a defect and is passed to the development team, the developer assigned is involved in *Make:Implement*.

In Fig. 2, we illustrate how practices relating to *Content* might be assigned to cells in the framework after interviewing the lead developer described in example 4 above.

4.2. Framework extensions

The basic framework illustrated in Fig. 1 enables us to build up a picture of the kinds of practices applied by organisations and their reported efficacy in meeting specific objectives. As a result of observations made during our trial study, we extended the framework in the following way. The extensions are used to both guide questions during framework use and to support data analysis. The framework has a number of possible applications (see Section 4.3) and the various applications will require the inclusion of specific extensions relevant for that application.

		Content
Define	Roadmap Scope	Policy of lead developer as primary interface to client Client meets with team to describe functionality Lead meets weekly with client to consolidate functionality Prototyping used to consolidate functionality Document functionality in Use Cases
Make	Design Implement	Design based on Use Cases Lead monitors functionality via informal meetings with team
Deliver	Integrate Release	Lead checks Use Case status Lead discusses Use Case status with client
	Support	

Fig. 2. Practices for the developer in example 4.

1. There appear to be a wide range of mechanisms for understanding what is to be done, for example, regular meetings, formal or informal documentation, informal discussions with team members, etc. We included for each cell a description for each of input (*In*), procedure (*Proc*) and output (*Out*). Mechanisms for input refer to the means by which participants get to know what they are supposed to do (as opposed to a list of documents required for the activity or a definition of the event that causes the activity to commence). Examples are 'formal document is given to me', 'I look on a wiki', 'we meet every day'. Mechanisms for procedures relate to tasks, tools and task-related communications mechanisms. Mechanisms for outputs relate to what happens to the result of the procedures, for example, 'I report to my manager', 'I put the information on a wiki', 'I deliver a test plan to my manager'.
2. In order to understand if a reported practice is consistently implemented, we have for each cell the concept of (*Same*). Should implementation of a practice differ across projects, teams or individuals, the *Same* value will be 'No'.
3. As we found during our trial that participants did not generally appear to distinguish between practices for new work, upgrades and defect management, we assumed in the framework similar practices for these but provided a different identifier for each to cover differences ('N' = new; 'U' = upgrades and 'D' = defects). If, for example, a practice is specific to new work, the captured practice is appended with the character 'N'.
4. As we are aiming to find out what really happens in software organisations, it is important that we distinguish between comments made by participants that relate to what they actually do and those that relate to 'how things are done'. For example, a manager might report strong process in place while the implementer might report otherwise. We need to know if there is conflicting information from implementers (implying lack of consistency) or agreement among implementers but management reports differ (implying management does not know what is going on). The protocol for use includes differentiating between general and experienced-based comments.

We illustrate these ideas in Figs. 3 and 4. In Fig. 3 we show a basic capture of a design review practice addressing the

	In	Content Proc	Out	Same
Make:Design	Consult wiki	Design review	Put on wiki	Yes

Fig. 3. Basic practice description.

	In	Content Proc	Out	Same
Make:Design	Consult wiki	Design review (N)	Put on wiki	Yes
	Ineffective — wiki is informal with no detailed information			

Fig. 4. More detailed practice description.

Content objective. The name of the practice in this cell indicates that design reviews are implemented and address product content, i.e. functionality. Practitioners involved understand what to do by consulting feature descriptions on a wiki and put a log of results on a wiki. All projects in the organisation carry out this practice in the same way.

For a more extensive capture, we expand (a) the *Proc* cell to include the information that the practice as relating to this objective takes place only for new work ('N') and (b) the *In* cell to state that the mechanism for knowing what to design is ineffective because the wiki generally does not contain all required information. This is shown in Fig. 4. If required for a specific study, a numeric scale might be applied to capture the degree of effectiveness (rather than textual comments).

This version of our framework does not deal explicitly with management practices, for example, risk, cost, knowledge management. Our hope is that practices will be reported if relevant for stated objectives. For example, if a project manager is asked about how content is assured in his or her projects, (s)he might report some communications mechanism involving the client. Quality objectives might be addressed by a formal acceptance procedure for sub-contractors or a risk assessment process for new technologies. Cost mechanisms will not be exposed when asking about objectives of content and quality, but would likely be reported when cost objectives are included in the future. In a similar way, if we are in the future investigating an objective relating to an objective 'developer satisfaction', practices such as 'Friday night drinks' might be revealed. We hoped to learn about some aspects of organisational maturity with the *Same* questions, i.e. is there a consistent, organisation-wide approach to practices.

4.3. Framework as research instrument

We stated in Section 1 that the contribution of this paper is a research framework that will help us investigate various aspects of software practices. In this Section, we expand upon this idea.

Our approach to understanding what goes on when developing computer software requires that we examine the actual practices that are carried out. This is in contrast with approaches that function at a higher level, and effectively 'wrap up' practices according to some paradigm. For example, the paradigm of 'software-as-manufacturing' has resulted in a wrapping up of practices into a set of processes and the paradigm of 'software-as-a-human-endavour' has spawned the agile methodologies. We submit that a focus on terms such as 'agile', 'traditional', 'process' and 'methodology' is not helpful and rather we must model at a more fine-grained level, i.e. at the practices level. Only in this way can we begin to compare these various paradigms and understand similarities, differences and gaps.

A model is generally understood as being some kind of abstraction of reality, one that allows us to view a real-world system in a way that is useful for some purpose. "Although wrong by definition, [models] are the wind that blows away the fog and cuts through the untamed masses of data to let us see answers to our questions" (Keller, 2006). When applying the framework described

above, we are declaring that we believe it may be useful to analyse a software development effort by:

- being clear about what are the main objectives to be met by the effort;
- categorising all practices that take place as addressing one or more basic functions;
- understanding exactly what are the practices project members carry out to effect the function and how effective these are at meeting the stated objectives;
- building up a picture of the development effort by asking the people who are actually doing the work.

We further believe that viewing a software development effort from the perspective embodied in the framework has potential for investigating the following kinds of research questions:

Is 'best practice' context dependent? The framework does not dictate what organisations *should* do but rather provides a mechanism for asking what works. We have an opportunity to investigate context-based best practices in an evidence-based way. For example, in the study reported in Section 5.2, we found that 'informal documentation' in conjunction with 'high collaboration' was reported as being the key mechanism for understanding what is to be done. We might now further investigate if this is a common approach for smaller groups and, whether it is, in fact, effective.

What are the risks of implementing a specific methodology? When we analyse the practices that make up a methodology by fitting these practices into the cells of our framework, do we find any gaps or badly-supported cells? Does the picture change when we consider the effectiveness of the practices in specific contexts? For example, in Section 5.3, we expose a gap in the *Roadmap* function for an 'ideal' eXtreme Programming (XP) approach.

Can we measure how cohesive a group is? We build our profile by entering data obtained from individuals. We would expect that several individuals will report the same practices for a given function and objective. However, we have found that it is often the case that individuals have varying perceptions about the *effectiveness* of practices. A development manager may believe the practice of product functionality documented on a wiki is extremely effective, while developers report that the wiki is never up-to-date, the information they need is too difficult to find and in the end they always have to find someone to ask. (Based on consulting work carried out by one of the paper authors, this lack of agreement is a relatively common occurrence.) If individuals report similar approaches to practices, we may infer that the organisation is functioning in a cohesive way. However, if individuals report disparate approaches, we might infer some lack of common vision or understanding within an organisation. The framework thus acts as a diagnostic tool.

Can we help a group streamline its processes? For each practice sub-category, we capture practice approaches in relation to specified objectives. We may notice some approaches that do not appear to contribute in a positive way to objectives and these may be viewed as possibly representing targets for software process improvement initiatives. For example, during our evaluation of the framework (see Section 5.2), we noticed developers often had to seek out information, either from a team member or from a wiki. This 'pull' approach to information elicitation may be inefficient and the possible effect on efficiency represents an area for future research.

Can we analyse how two groups differ? Two organisations claiming to implement an agile approach or a practice, for example, pair programming, may do so in very different ways, because of different contexts. Even if the actual procedures are the same and comply with recommendations, the effects on objectives may be

quite different. For example, organisation A may always use experienced developers whereas B may often assign inexperienced developers. A is likely to be effective in meeting quality objectives whereas B is not. Further, when talking with one of A's developers, we may learn that 'Fred never listens to what I say and wants to do it all himself and I think we actually do very little quality checking'. With our framework we specifically ask individuals about effectiveness and so we have an opportunity to investigate the effects of different contexts on productivity.

Is it possible to categorise organisations? At the moment, the industry categorises software organisations according to characteristics such as 'organisational maturity' or 'traditional versus agile'. The first appears to be of low interest to many (especially smaller) organisations and the second is so ill-defined as to be of little use. Our research framework presents us with the possibility of uncovering new ways of categorising, i.e. according to approaches to, and perceived efficacy in, meeting software objectives. We are in the process of further investigating this possibility of profiling organisations in this way with a survey based on the framework and involving 198 participants from 51 NZ software organisations.

Do comparison studies actually compare the same thing? One issue with gathering evidence relating to specific practices is that often practices with the same name are carried out in quite different ways. For example, in an attempt to compare three studies into pair programming, (Kirk, 2007) exposed some key differences between the practices described in the studies that effectively rendered comparison impossible. A first step towards measuring is to understand what is actually taking place. We have an opportunity to find out what a practice *really* entails and thus support evidence accumulation.

When using the framework, the actual information sought and level of detail will depend upon what the question is. For example, when exploring group cohesion, we are interested in effectiveness ratings as we want to expose differences in perceptions. When analysing differences between groups, we need to find out what the practices actually entail as we suspect different people mean different things. When analysing an 'ideal' process or methodology for risk, i.e. where we assume the claims of the methodology are met, we simply need check that each cell in the framework is addressed by some activity. For risk assessment in a real context, we would like to know, in addition to how many different practices relate to each cell, whether these are effective in the specific context.

4.4. Profiling with the framework

The framework described above supplies a structure for understanding practices in organisations. In this Section, we show how the profile for an organisation can be built up based on individual viewpoints about practice importance and efficacy in relation to a given objective.

The importance of acknowledging the perspective of individuals regarding process efficacy has long been established, and has roots in the 'continuous improvement' paradigm emerging from Japan in the 1940s. In more recent years, there has been a renewed interest in approaches such as the Repertory Grid Technique, a technique that is based on personal construct theory and the notion that individuals' cognition can affect outcomes (Edwards et al., 2009; Tan and Hunter, 2002). We leverage the strengths of these approaches by basing our profile on information elicited from a range of individuals from the group to be studied. The expectation is that, as we enter information from different people into the cells of our framework, a picture will emerge that will help us answer the questions we were asking, or will highlight new

		Content	Quality
Define	Roadmap	1a	1b
	Scope	2a	2b
Make	Design	3a	3b
	Implement	4a	4b
	Integrate	5a	5b
Deliver	Release	6a	6b
	Support	7a	7b

Fig. 5. Typical coding scheme for elicited information.

questions to ask. In this Section, we describe the mechanics of using the framework.

The approach for applying the framework involves the following steps:

1. Identify with the manager the objectives and scope of the study. For example, a manager may want to study a 'typical' project to expose ineffective practices or may want to explore whether the employees in his organisation function in a cohesive way.
2. Identify the main objectives. The manager must be clear about which objectives matter the most. For example, is product reliability and security more important than delivering all expected functionality?
3. Establish the detail of information required and the coding scheme to be used (an example is presented below).
4. Obtain from the manager a list of employees to interview. Employees should cover as wide a range of roles and experiences as possible.
5. Hold interviews, coding each relevant piece of practice information, for example, using the coding scheme depicted in Fig. 5.
6. Collate all information into the cells of a spreadsheet and analyse.

A typical basic coding scheme for a study with objectives *Content* and *Quality* is shown in Fig. 5.

The mechanism for interviewing practitioners involves, for each functional sub-category, establishing if the practitioner is active in this sub-category and then asking questions relating to the sub-category. The information provided is captured in note form on paper with comments numbered to reflect their 'meaning' in terms of the defined coding. Questions are based directly on the framework and we illustrate with an example involving a developer whose main task is low level design and coding. We have established that (s)he is mainly involved in *Design*, *Implement* and *Integrate*, has no involvement in roadmapping or project scoping, does not deal with the product in the target environment (and so is not involved in support) but does release patches directly to the client, and so has a role in *Release*.

For this practitioner, the relevant framework rows in Fig. 5 are rows 3, 4, 5 and 6. Questions are asked for each relevant cell and relate directly to the extended framework as described in Section 4.2. In Appendix A, we illustrate the interview procedure for cells 4b and 6b, i.e. how the developer addresses ensuring the quality of implemented software and how (s)he makes decisions about quality when releasing patches.

Information that is noted includes the practice name and description along with some observations about the perceived effect on the objective and rationale for the observation. The extent of information captured will be dependent upon the reason for using the framework. The process thus involves first ensuring *all* relevant activities are reported for each participant and then collating the comments in a spreadsheet. Information may then be analysed according to the form of the research that is being carried out.

We reiterate that, because we are aiming to understand what actually happens, there can be no separation of practice and context. Unless we are profiling an ideal process or methodology (see Section 5.3), activities such as, for example, 'Prototype' and 'Pair Programming' are understood only when we know who was involved and their characteristics. Was a prototype whose aim was to consolidate functionality actually shown to the client, or was it merely used as a basis for development? In the former case, it probably served as an effective activity for the *Content* objective, whereas in the latter case it was probably extremely ineffective. Is a large group who practice pair programming less effective than a small one? If we apply our approach to individual members of the large group, we might find that there is disagreement about what the practice actually entails, or that different people have different experiences when applying the practice. If we have questions about the efficacy of pair programming in a large group, we must capture the experiences of individuals within the group and then examine the experiences for consistency or otherwise. The (in)consistencies in responses along with gaps are an integral part of profiling the larger group. For example, if those with role 'Manager' report that a practice is extremely effective and those with role 'Developer' mostly report otherwise, we can infer some lack of common understanding, either accidental or intended, between management and development and need to explore this before we can study pair programming effectiveness in large groups.

5. Evaluating the framework

We have introduced a research framework and made claims that the framework will be useful for supporting various kinds of research in software development. We must now provide evidence to support these claims. Dawson et al. (2004) suggests that, for complex, human-intensive systems, there is a case for accumulating many little evidences of different kinds and they reference the fields of medicine, law, industrial engineering and knowledge management to support this viewpoint. This approach compares with that of other researchers who draw from the fields of psychology and sociology to suggest the accumulation of an 'evidence portfolio' comprising case studies, anecdotes, surveys, expert opinion and controlled experiments (Kitchenham et al., 2005; Weaver et al., 2005).

As a means of presenting evidence to support our claims, we have chosen an approach called *argumentation* along with an established notation, *goal structure notation (GSN)*, for structuring and capturing arguments. This approach has been used for many years in the safety critical domain and has more recently been applied in the software domain (Weaver et al., 2005). *Argumentation* is "an approach which can be used for describing how evidence satisfies requirements and objectives" (Weaver et al., 2005). The use of a suitable notation such as GSN helps researchers to easily identify what evidence is required and helps stakeholders see at a glance what is the 'evidence coverage'. We have chosen this strategy for two reasons. The first relates to the fact that, as we are evaluating a research framework to be used for answering different kinds of questions, the breadth of evidence required is large. The argumentation approach helps organise and display evidence such that both evidence and lack of evidence are relatively easy to see. The second reason relates to the idea presented above that, for people-intensive systems, accumulating a portfolio of different kinds of evidence is appropriate. The argumentation approach provides for individual items of evidence and so is suitable for the portfolio idea.

In this Section, we present the evidence map for our framework and then discuss in greater detail the three existing pieces of evidence, relating to gaining insights into possible context-dependent

success mechanisms in three New Zealand organisations and to exposing risk in the eXtreme Programming (XP) methodology.

5.1. Evidence map

In Fig. 6, we show the evidence map relating to our research framework. With GSN, goals are presented as rectangles, arguments as parallelograms and evidence as circles. The goal hierarchy is thus a structure showing how goals are addressed by arguments and how arguments are supported by evidence. Arguments for which no evidence exists, i.e. undeveloped goals, have an attached diamond.

Our top-level goal (Goal 1) is to accumulate evidence relating to our research framework. For each of the possible research questions to be supported by the use of the framework (see Section 4.3), we have created an argument and we have expanded each argument as one or more sub-goals. Sub-goals at this point are minimal and will multiply as the research matures further, possibly creating more levels in the hierarchy. For sub-goals 1.1.1, 1.2.1, 1.2.2 and 1.4.1, we have referenced our current evidence. Evidence for 1.1.1 and 1.4.1 relate to an organisational case study. The evidence is labelled as evidence circle 1 and is presented in Section 5.2. Evidence for goal 1.2.1 (evidence circle 2) relates to a paper study of an ideal XP process (presented in Section 5.3) and evidence for goal 1.2.2 (evidence circle 3) is a 'what-if' consideration of XP in context (see Section 5.4).

Note that this 'evidence map' captures breadth of evidence rather than depth, i.e. the strength of individual pieces of evidence must be obtained from studying the actual evidence. The strength of the map is rather to provide a quick idea of what evidence exists and easy access to that evidence (Weaver et al., 2005).

As evidence accumulates, we may find that our framework is useful for some kinds of research questions and not for others. We may, of course, also identify other areas of investigation we believe may be supported by the framework and add these to the map. For example, for Goal 1.2.2, we clearly need to accumulate evidence gained by studying a number of standard methodologies as implemented in different kinds of organisations with different objectives.

5.2. Evidence 1: Organisational practices in context

In this Section, we describe how our research framework was applied in the three New Zealand organisations that had agreed to participate in our original study. We overview the characteristics of these organisations below.

Organisation A delivers health information systems to both local and global markets. It originated ten years ago and now has 250 staff, with approximately 100 involved in development. All development takes place at a single location. The culture was reported by the manager as informal with a flat management structure and focus on personal communications rather than a strict adherence to a specific development methodology. We interviewed a product manager, a design engineer, a developer and a test engineer.

Organisation B delivers business design software and services to a range of clients in the private and public sectors. There is a core team of approximately six and development is carried out by a small number of contractors who have been involved since inception. The group reports being informal and focuses on the use of technology to support operations. Again, the manager reported no adherence to a specific methodology. We interviewed the CEO, a senior manager and a QA manager. One potential participant, a developer, did not respond to our invitation to participate.

Organisation C produces network monitoring software for both public and private sectors. There are approximately 40 development staff spread over two locations, with quality assurance personnel located at one of these. The organisation is

undergoing growth and has recently restructured, with plans to improve operational effectiveness by introducing formality into all software processes. Development processes are thus in flux. We interviewed a senior product manager, a development manager and a QA manager.

Although each of the participating organisations had an interest in process improvement, only organisation C was actively involved in an improvement initiative at the time of the interviews. This initiative was informal in that its direction was set by the senior manager and based on his previous experience.

We held informal interviews using the kinds of questions presented in Appendix A. We applied a coding scheme as suggested in Section 4.4 to match information about practices to the appropriate cells in the framework. In addition to questions guided by the framework, we also asked each participant what (s)he believed were the main contributors to success and which practices were problematic. Comments were recorded as free-form sentences.

We were able to complete all interviews within an hour. As the questions directly related to the framework, we were able to code comments easily.

Collected data was collated in a spreadsheet. Comments for each code were placed in the appropriate cell and colour-coded according to the owner of the comment. Comments about practices actually carried out by the participant were in plain font and those reporting what others did were italicised.

The cells were then examined for any potential insights or observations of interest. We observed the following:

- For cells with several comments, the comments were similar for some organisations and not for others. Comments for Organisation A tended to exhibit consistency in both cells relating to inputs and those relating to procedures. For example, comments of the form 'I find out about the product from regular, informal chats with team members', 'I talk to the analyst when I'm unsure', 'we get it right because we have subject area experts on the team' were common to all roles interviewed. This was not the case for organisations B and C. Although we do not have sufficient data to infer reasons, we suggest that such inconsistency represents a serious symptom. When involving management and employees (as in the case with B and C), the implication is that managers do not know what is going on. When involving employees only, the implication is a lack of a common vision and culture.
- Two of the three organisations reported use of informal documentation for understanding, informal processes for activities and a reliance on expertise, i.e. an 'informal' culture. However, one organisation also reported a reliance on extensive communication and collaboration within the group while staff in the second organisation tended to work in a more individual way. We wonder if the existence of communication and collaboration in some way 'strengthens' the organisation and we wonder if there is a possibly useful distinction between 'strong' and 'weak' informal cultures.
- All of the participant organisations reported a dependence on Pull mechanisms, i.e. the participant has to actively search for information about what needs to be done. There is a possibility that this implies an inefficiency, for example, as individuals must spend time finding the relevant information.
- For organisations that appeared to be low in agreement about practices, the employee participants reported higher numbers of issues.

It is clear that, despite interviewing only a small number of people from each organisation, basing questions on our framework allowed us to make a number of interesting observations in a

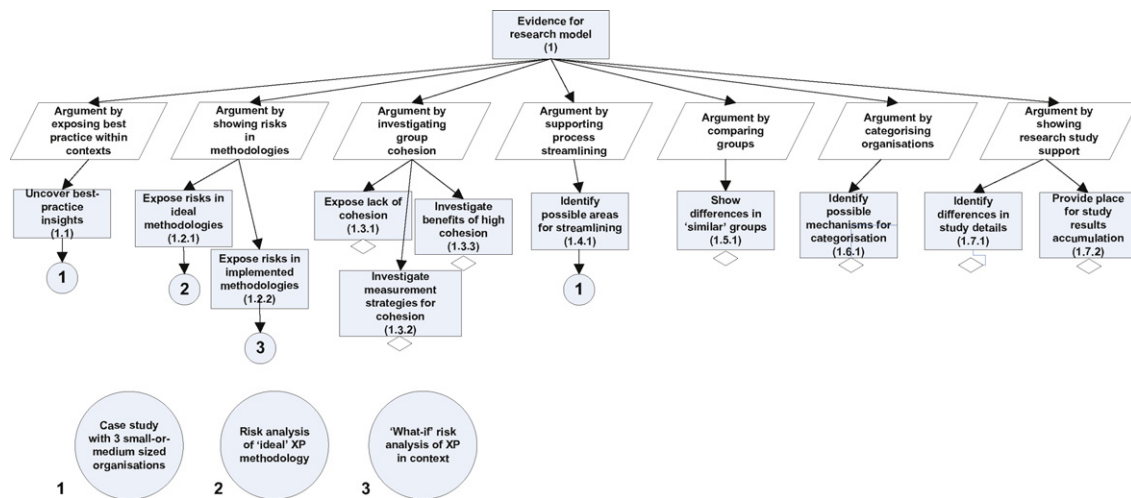


Fig. 6. Evidence map for framework.

relatively short space of time. Each of the observations mentioned above represents an area for further investigation.

5.3. Evidence 2: Risks in idealised XP

In this Section, we use our framework to capture the twelve XP Practices as cited by Beck (2000). XP was selected for exploring risk because its practices are well-defined but do not obviously fit into a standard process paradigm. For example, what is the process that is described by *Metaphor*? We hoped the attempt to capture in our framework would provide an interesting and useful addition to our evidence portfolio.

As in the rest of this paper, we restrict objectives to functionality and quality. We first take the perspective of Beck, i.e. we capture XP characteristics as claimed rather than as experienced by individuals in specific contexts. We then experiment with different environmental characteristics.

Planning game is the main mechanism for participants in an XP project to help them understand what is to be made. As all project roles participate, assuring correct functionality is addressed in *Scope*, *Design*, *Implement*, *Integrate* and *Release*. The practice does not address *Roadmap* (long term, strategic planning) or *Support* (strategies relating to the product in the target environment).

Small releases introduces constraints on the *Release* interface between client and development organisation by defining release frequency and providing an opportunity for the client to monitor content and quality. The practice itself does not involve feedback and so no other categories are involved.

Metaphor supports a common understanding between client and developer. The *Content* objective is supported for all categories other than *Roadmap*.

Simple design is an instruction to developers to keep code design as simple as possible, with a view to minimising errors. The *Quality* objective is addressed in *Design* and *Implement*.

Testing has two aspects. The first is the instruction to a developer to continuously write unit tests. This supports *Quality* in *Design* and *Implement*. The second is the need for a client to test all delivered software. The aim is not to accept or reject the release, i.e. the practice does not represent a constraint on the *Release* category as the software is delivered no matter what, but rather the aim is to provide feedback to the team. This means increased understanding of the product for both team members and client and so both objectives are addressed in all categories other than *Roadmap*.

Pair programming mandates a specific procedure involving two developers who act as checks for each other. Both objectives are addressed in *Design* and *Implement*.

Collective ownership mandates that all code may be changed by any developer. The intent is to improve code quality. The *Quality* objective is addressed in *Design* and *Implement*.

Continuous integration demands that code is integrated after every change with a view to quickly identifying and resolving defects. The *Quality* objective is addressed in *Integrate*.

40-h week has the aim of ensuring team members remain motivated and able to work in an effective way. This potentially affects both objectives in *Make* and *Deliver*.

On-site customer acts as an information source for resolving product-related uncertainties. Both objectives are addressed in *Make*. The increased product understanding of the customer also potentially addresses issues relating to the product in the target environment, i.e. *Support*.

Coding standards specifies that code must comply with an agreed standard. The *Quality* objective is addressed in *Design* and *Implement*.

In Fig. 7, we have inserted these practices into the appropriate cells in our framework.

One observation from our capture of a 'theoretical' XP approach is that the *Roadmap* function is not addressed. This highlights a potential issue for product-line contexts, i.e. the function must be addressed outside the XP methodology. This outcome is to be expected, as XP was not designed to address roadmapping. The point is, of course, that the framework successfully highlighted the expected gap.

A second observation is that most of the practices address both *Content* and *Quality* objectives across a range of function categories. This is a result of the managed information sharing in XP where everyone, including the client, attends regular *Planning game* meetings and a client is always available for feedback. We may ask what would happen if one of the project roles, for example, test, was missing from these meetings.

5.4. Evidence 3: Risks in XP in context

We now experiment with the contextual nature of organisations by considering a context in which a capable client representative is not available. When we remove the contributions of the client-dependent practices (*Planning game*, *Metaphor*, *Client testing* and *On-site customer*), we have the framework shown in Fig. 8. Although

		Content	Quality
Define	Roadmap Scope	Planning game Metaphor Client testing	Client testing
Make	Design	Planning game Metaphor Client testing Pair programming Collective ownership On-site customer 40-hour week	Simple design Unit testing Client testing Pair programming Collective ownership On-site customer 40-hour week Coding standards
	Implement	Planning game Metaphor Client testing Pair programming Collective ownership On-site customer 40-hour week	Simple design Unit testing Client testing Pair programming Collective ownership On-site customer 40-hour week Coding standards
	Integrate	Planning game Metaphor Client testing Continuous integration On-site customer 40-hour week	Client testing Continuous integration On-site customer 40-hour week
Deliver	Release	Planning game Small releases Metaphor Client testing 40-hour week	Small releases Client testing 40-hour week
	Support	Client testing Metaphor On-site customer 40-hour week	Client testing On-site customer 40-hour week

Fig. 7. XP practices as captured in the research framework.

the *Make* function continues to be well-addressed, we now observe risk in the product definition and delivery functions. There are no mechanisms for roadmapping and scoping the product, and decisions relating to releasing the product to the client are addressed by only *Small releases*, i.e. efficacy is no longer supported by high product knowledge within the development organisation. The consequences indicated by this analysis relate to possible problems in identifying and managing requirements and in establishing release content and quality. Indeed, recent research by Hoda et al. (2011) indicates that inadequate customer involvement may result in such

issues. Our framework has helped us expose another valid area of risk.

Some other contexts that might be worth modelling include inexperienced development staff (reduced effectiveness for *Pair programming* and *Collective ownership*, introducing risk in the *Make* function) and large products that take some time to build (now no mechanisms in place for assuring an effective integration function). It might also be fruitful to introduce new objectives. For example, how does an XP approach address a *Cost* objective?

6. Discussion

The framework we have described has been developed with different uses in mind. Primarily we would like to answer the kinds of questions described in Section 4.3. To do so requires capturing the right kind and amount of information about how an organisation develops software with minimal cost. Ultimately, how successful our framework is will only be determined with experience. However we believe the evidence we have presented in this paper gives us reason to believe it can be successful.

The evidence we have presented in Section 5 addresses the main questions about the usefulness of our framework. These are:

1. Is the framework complete, that is, are there no gaps in the framework so that every aspect of how an organisation develops software can be represented?
2. Is the information captured by the framework sufficient for answering the research questions we care about?
3. Is the framework lightweight, repeatable, and flexible?

We address the first question in two ways. First we applied the framework to an established process, reasoning that if we could not represent such a process that would indicate a flaw in the framework. We chose XP for this test, because we developed the framework based on our experience with more traditional processes, and so reasoned that if there were any gaps they would be exposed by examining an agile process.

The second test was to use the framework for real. Our reasoning was that organisations may have practices not described in formal processes, and so this would test our framework for such situations.

The second question is more difficult to address without actually performing studies based on the kinds of questions described in Section 4.3, but we do not want to do such (expensive) studies without some assurance that the framework will be successful. Instead, we tried to answer a more nebulous question—Can we learn something interesting from the information captured by the framework? If the answer to this question is yes, then that gives us confidence to use the framework in a more expensive study.

Again both the application to a formal model and use with real organisations provides evidence to support our belief that the framework will be successful. From the results for the organisations, we discovered a potentially interesting pattern for communication that may impact the success of organisations. From the ‘what-if’ application to the results for XP, we were able to identify risks that occur if the context for the use of XP is not as expected.

For question 3, using the framework in industry allowed us to test whether the framework was lightweight enough to be used with minimal cost, and whether its use was acceptable to organisations. Because different people answered differently to the questions we posed, this also allowed us to test the framework’s flexibility. Repeatability is supported by the fact that the researcher asks the same kinds of questions each time (see Appendix A).

		Content	Quality
Define	Roadmap Scope		
Make	Design		Simple design Unit testing
		Pair programming Collective ownership	Pair programming Collective ownership Coding standards
	Implement		Simple design Unit testing
		Pair programming Collective ownership	Pair programming Collective ownership Coding standards
	Integrate	Continuous integration	Continuous integration
Deliver	Release	Small releases	Small releases
	Support		

Fig. 8. XP practices without effective client participation.

The apparent success of our framework could be due to the number and choice of organisations we used it for, and the number and choice of participants. It is unlikely that our sample is representative of all software development organisations, or even a large subset, but getting a representative set will take time and experience, and the evidence we have gathered gives us the confidence that it is worth spending this time.

A threat to validity of the organisational case study is that we were dependent on managers suggesting individuals to include and so we may have interviewed only individuals with a common 'story'. This could mean that our framework is more difficult to apply effectively when a more realistic cross-section of employees is involved. We mitigated against this risk by asking managers for names of potential participants only after the manager had a good idea of what we were trying to achieve. This appeared to work, as two of the three organisations exhibited dissenting viewpoints. Other threats to validity are the small number of organisations and participants included and issues of objectivity raised by a single person carrying out interviewing, coding and analysis. As the case study was intended as being exploratory in nature, we do not believe these threats pose great risk. The fact that some potentially useful insights were gained despite low numbers of participants may be viewed as a likely strength of the framework.

Threats to validity for the idealised XP study are that analysis of the XP practices in relation to the framework was subjective and that it is known that XP does not deal with planning.

A limitation of the research framework relates to the 'general' information about the organisation and its products. The participant template has a focus on the project, i.e. asks about development and planning and does encourage some organisational information by means of the 'same each time' questions. However, in one interview, the participant had a role that included marketing and provided much information about the product's history, for example, which products had been acquired and rebranded. This information is possibly important but there was nowhere to capture this in the framework in its current form.

7. Conclusions and future work

We have developed a research framework to support studies relating to software practices. Our approach differs from others in that we are trying to avoid any preconceptions of how software *ought* to be developed, and so are explicitly not evaluating against a specific formal model or set of practices. Our motivation for this approach is the evidence in the empirical software engineering literature that many organisations have not adopted the formal models.

Our research framework was created as a result of experiences during a trial to test an approach based on a standard assessment model. The trial, involving 4 staff from two separate groups in an organisation, exposed the need for a new approach. The result is our research framework. This is based on the viewpoint that practices exist to meet specified objectives and on the understanding that we must understand practices as actually implemented in the context under study. The framework provides a means of describing how organisations implement software practices to meet specific objectives.

The research question addressed in this paper relates to the usefulness of the framework for yielding insights for investigation. We have presented three studies as evidence to support this question. The first was an exploratory case study involving 11 staff of three NZ software organisations. Questions asked were based on the framework. We found that we were able to ask all relevant questions within an hour and we could easily capture and classify information provided by participants. For this kind of study, the framework

supports a lightweight and flexible approach. Although only a small number of participants was interviewed, we exposed some possible mechanisms for success, for example, the use of informal practices in conjunction with strong communications. We also exposed the use of a 'pull' mechanism for understanding requirements and suggest this represents a possible area of inefficiency within the organisations. We note that these insights result directly from studying the data collected when applying the framework and required neither a targeted approach nor specialised interview and analysis techniques.

In the second and third studies, we applied the framework to an idealised and 'what-if' XP process. We exposed a gap in support for long-term planning in the idealised methodology and yielded some understanding of vulnerabilities resulting from 'non-ideal' contexts. For example, when experimenting with removing client availability, we noted a gap in practices for defining and delivering software products.

The successful application of the framework in the three studies described suggest that the framework does adequately capture what happens in an organisation in a way that supports answering questions relating to (a) project risk caused by gaps in practices and mis-matches in project members perceptions of practices, (b) comparing organisations by understanding contextual effects on practices, (c) streamlining by identifying practices that do not contribute effectively to a stated objective and (d) categorising organisations according to practices implemented.

The framework presented here allows us to see software development efforts in a different way, and presents many avenues for future investigation (see Section 4.3). We will follow two paths in the immediate future. We are first exploring the possibility of categorising organisations according to the practices implemented. Our approach is to carry out a large-scale on-line survey using a questionnaire instrument based on the framework. The survey stage is complete, with 198 participants from 51 NZ organisations, and we are now in the analysis stage. We next plan to investigate the 'agile' nature of organisations by using the framework to compare standard agile processes and organisations claiming to be 'agile'. We will apply a case study approach for this investigation. The research will take some years to mature and requires the contribution of others for effective advancement.

Role of the funding source

This research is funded by the New Zealand Foundation for Science and Technology Research (FRST). The funding source had no involvement in study design, collection, analysis and interpretation of data, report writing or in the decision to submit the paper for publication.

Acknowledgements

This research is part of the Software Process and Product Improvement (SPPI) project, funded by the New Zealand Foundation for Science and Technology Research (FRST). We would like to thank the organisations and individuals who agreed to take part in our study.

We would also like to thank the anonymous reviewers for the paper. Their thoughtful feedback was extremely helpful to us and has thus contributed greatly to the quality of the paper.

Appendix A. Examples of interview questions

We illustrate the interview procedure for eliciting information for cells *Implement:Quality* and *Release:Quality*, i.e. how the developer addresses ensuring the quality of implemented software and

how (s)he makes decisions about quality when releasing patches. We reference the questions back to the extensions described in Section 4.2.

A.1. Questions for Implement:Quality

We would like to know how you address the quality of the software you produce. Please tell us when you are involved in a practice yourself and when you are telling us about a practice carried out by someone else in your group (Framework extension 4, coded as *O* if report about others).

1. First of all, tell us how you get to know what is expected in the way of software quality. For example, are the expectations written down somewhere or do you find out by talking to people? (Framework extension 1, coded as *In*.)
2. Now tell us if you have anything in place for ensuring the quality of the software you implement. For example, do you write unit tests or run tests written by others? Does someone review your code or your tests? (Framework extension 1, coded as *Proc*.)
3. How do others get to know about the quality of your software? For example, do you log formal review sign-offs or is it assumed that tests have been run and passed? (Framework extension 1, coded as *Out*.)
4. Is this the same for new development and maintenance work? If not, tell us how these differ? (Framework extension 3, coded as *N*, *U* and/or *D*.)
5. Do you always do things this way? (Framework extension 2, coded as *Same*.)
6. How well do you think this works—is what happens effective, or are there any problems? What are they? Why?

A.2. Questions for Release:Quality

When you release patches to the client, we would like to understand the kinds of decisions you make about the quality of the patch. Please tell us when you are involved in a practice yourself and when you are telling us about a practice carried out by someone else in your group (Framework extension 4, coded as *O* if report about others).

1. First of all, tell us how you get to know what level of quality is appropriate for the patches you send to the client. For example, do you check documents stating quality expectations or perhaps you talk it over with a colleague? (Framework extension 1, coded as *In*.)
2. Now tell us how you actually go about releasing patches. For example, do you have a written process or do you make the decision about how well to test based on your experience? Do you always attach a release document? How is the patch distributed? (Framework extension 1, coded as *Proc*.)
3. How do others get to know about the release decisions for the patch? For example, do you log in a release log? (Framework extension 1, coded as *Out*.)
4. Is the process the same for enhancements and defects? If not, tell us how these differ? (Framework extension 3, coded as *N*, *U* and/or *D*.)
5. Do you always do things this way? (Framework extension 2, coded as *Same*.)
6. How well do you think this works—is what happens effective, or are there any problems? What are they? Why?

References

- Bach, J., 1995. Enough about process: what we need are heroes. *IEEE Software* 12, 96–98.
- Basili, V.R., Caldiera, G., Rombach, H.D., 1994. The experience factory. In: Marciniak, J.J. (Ed.), *Encyclopedia of Software Engineering*, volume 1. John Wiley and Sons, Ltd..
- Baskerville, R., Pries-Heje, J., Madsen, S., 2011. Post-agility: what follows a decade of agility? *Information and Software Technology* 53, 543–555.
- Beck, K., 2000. *Extreme Programming Explained—Embrace Change*. Addison-Wesley, United States of America.
- Cater-Steel, A., Toleman, M., Rout, T., 2004. After the assessment: actions and reactions of 22 small Australian firms. In: 4th International SPICE Conference on Process Assessment and Improvement—SPICE 2004.
- Chrissis, M.B., Konrad, M., Shrum, S., 2007. *CMMI Second Edition Guidelines for Process Integration and Product Improvement*. Addison-Wesley, Massachusetts, USA.
- Coleman, G., 2005. An empirical study of software process in practice. *Hawaii International Conference on System Sciences* 9, 315c.
- Cusumano, M., MacCormack, A., Kemerer, C., Crandall, B., 2003. Software development worldwide: the state of the practice. *IEEE Software*, 28–34.
- Dawson, R., Bones, P., Oates, B.J., Brereton, P., Azuma, M., Jackson, M.L., 2004. Empirical methodologies in software engineering. In: *Proceedings of the Eleventh Annual International Workshop on Software Technology and Engineering Practice*, IEEE.
- Dutta, S., Lee, M., Van Wassenhove, L., 1999. Software engineering in Europe: a study of best practices. *IEEE Software* 16, 82–90.
- Edwards, H.M., McDonald, S., Young, S.M., 2009. The repertory grid technique: its place in empirical software engineering research. *Information and Software Technology* 51, 785–798.
- El Emam, K., Garro, I., 2000. Estimating the extent of standards use: the case of ISO/IEC 15504. *Journal of Systems Software* 53, 137–143.
- Hoda, R., Noble, J., Marshall, S., 2011. The impact of inadequate customer collaboration on self-organizing agile teams. *Information and Software Technology* 53, 521–534.
- ISO (International Standards Organisation), 2004–2006. *ISO/IEC 15504: Information Technology—Process Assessment (Parts 1–5)*, ISO.
- Keller, D.K., 2006. *The Tao of Statistics*. Sage Publications Inc.
- Kirk, D., 2007. *A Flexible Software Process Model*, Ph.D. Thesis. University of Auckland, Auckland, New Zealand.
- Kirk, D., MacDonell, S., Tempero, E., 2009. Modelling software processes—a focus on objectives. In: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, Session: Onward short papers, ACM Press, pp. 941–948.
- Kirk, D., Tempero, E., Submitted for publication. Using an Assessment Model for Understanding Practices in SMEs: An Experience Report.
- Kitchenham, B., Budgen, D., Brereton, P., Linkman, S., 2005. Realising evidence-based software engineering. In: *Realising Evidence-Based Software Engineering Workshop 2005*, Workshop Co-located with ICSE 2005, Keele University, St. Louis, Missouri.
- Laporte, C.Y., Alexandre, S., O'Connor, R.V., 2008. *A Software Engineering Lifecycle Standard for Very Small Enterprises*. Springer-Verlag, Berlin, Heidelberg, pp. 129–141 (volume 16 of *Communications in Computer and Information Science (CCIS)*).
- Livari, J., Livari, N., 2011. The relationship between organizational culture and the deployment of agile methods. *Information and Software Technology* 53, 509–520.
- MacCormack, A., Kemerer, C., Cusumano, Crandall, 2003. Trade-offs between productivity and quality in selecting software development practices. *IEEE Software* 20, 86–93.
- McCaffery, F., Pikkariainen, M., Richardson, I., 2008. AHAA—Agile, hybrid assessment method for automotive safety critical SMEs. In: *Proceedings of the 30th International Conference on Software Engineering (ICSE'08)*, ACM Press, Leipzig, Germany, pp. 551–560.
- MMoRST (New Zealand Ministry of Research, Science and Technology), 2009. *Knowledge Intensive Service Activities in the New Zealand Software Industry*.
- Munch, J., 2005. Goal-oriented composition of software process patterns. In: *Proceedings of the 6th International Workshop on Software Process Simulation and Modeling (ProSim'05)*, Fraunhofer IRB, St. Louis, Missouri, pp. 164–168.
- Petersen, F., Ivarsson, M., Gorschek, T., Ohman, P., 2008. A practitioner's guide to light weight software process assessment and improvement planning. *Journal of Systems and Software* 81, 972–995.
- Sison, R., Jarzabek, S., Hock, O.S., Rivepiboon, W., Hai, N.N., 2006. Software practices in five Asean countries: an exploratory study. In: *ICSE '06: Proceedings of the 28th International Conference on Software Engineering*, ACM, New York, NY, USA, pp. 628–631.
- Staples, M., Niazi, M., Jeffery, R., Abrahams, A., Byatt, P., Murphy, R., 2007. An exploratory study of why organizations do not adopt CMMI. *Journal of Systems and Software* 80, 883–895.
- Tan, F., Hunter, M.G., 2002. The repertory grid technique: a method for the study of cognition in information systems. *MIS Quarterly* 26, 39–57.
- Weaver, R., Despotou, G., Kelly, T., McDermid, J., 2005. Combining software evidence—arguments and assurance. In: *Realising Evidence-Based Software Engineering Workshop 2005*, Workshop Co-located with ICSE 2005, Keele University, St. Louis, Missouri.

Yin, R.K., 1994. *Case Study Research: Design and Methods*. Sage Publications, Inc., California, USA.

Diana Kirk is a Research Fellow in the Department of Computer Science at The University of Auckland. Diana was awarded a Masters in Computer Science from the University of Edinburgh and a Doctorate from the University of Auckland in 2007. Her industry experience spans over twenty years and includes technical programming, co-directing an emerging company, project management, software quality management and consulting. Diana's main interests relate to maximising effectiveness in the processes applied during software projects.

Ewan Tempero is an Associate Professor in the Department of Computer Science at The University of Auckland, New Zealand. He graduated from the University of Otago, New Zealand, with a B.Sc., (Honours) in Mathematics in 1983 and received his Ph.D. in Computer Science from the University of Washington, USA, in 1990. He has published over 140 papers in journals and internationally-refereed conferences, mainly in the areas of software reuse, software tools, and software metrics. His current research is developing metrics for measuring the quality of software designs.